什么是 Skills?

我们从最基础的大模型、Prompt，一路学到了 Agent、Function Calling、MCP，前面这些技术在咱们之前的「智能OnCall项目」中都有用到。

不过，从 26 年初开始，AI 圈子又出了个叫 **Skills（技能）** 的新东西。

当时我们在做智能 OnCall 项目时，还没有 Skills 这个概念，自然项目也就没有用到。不过今天，我还是得专门带大家来学习一下 Skills 这个新玩意，因为它现在真的太火了，现在的 AI 岗位面试几乎逢考必问！

很多同学在看各种 AI 框架源码时，经常被这几个词绕晕：「它们到底是个啥？谁先出谁后出？有啥区别？」

别慌，今天咱们就泡杯茶，用最接地气的话，按照技术演进的真实时间线，把这三个概念从头到尾盘一盘。

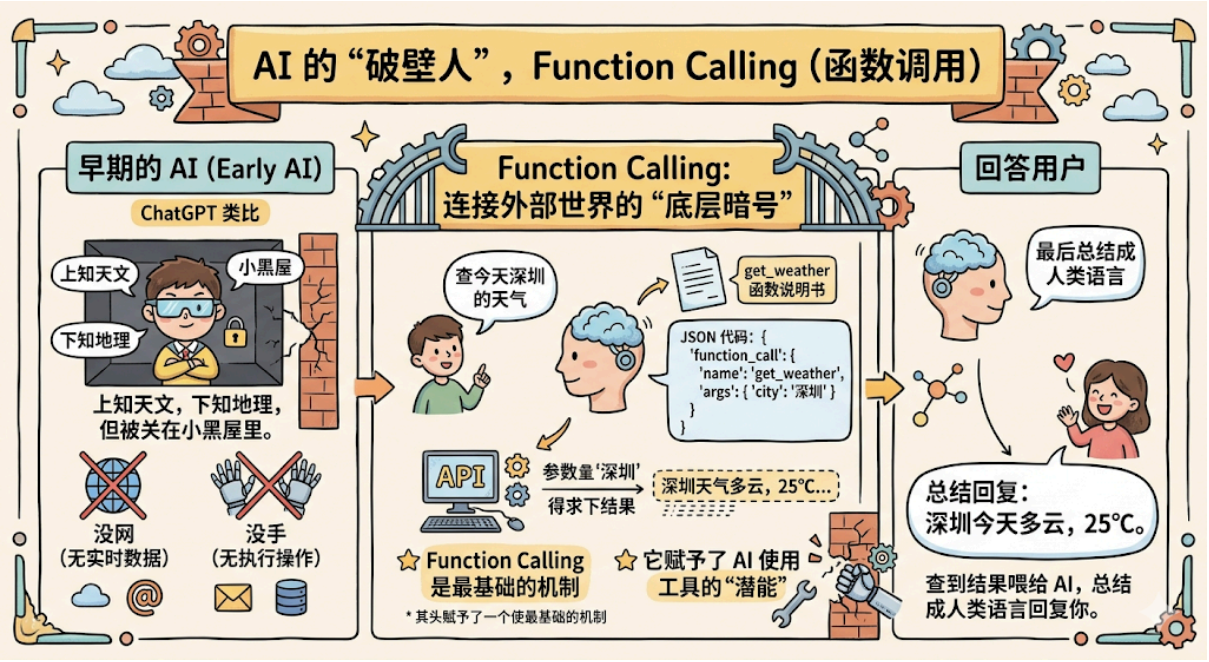
第一阶段：AI 的「破壁人」，Function Calling（函数调用）

咱们先回想一下早期的 ChatGPT。

那时的它就像是一个**被关在小黑屋里的超级学霸**：虽然上知天文下知地理，但他没网（没法查实时数据），也没手（没法帮你真正去执行操作，比如发邮件、查数据库）。

为了让大模型能连接外部世界，**Function Calling** 诞生了。

用白话说，这就是**大模型和外部世界打交道的一种「底层暗号」**。



你想让 AI 查今天深圳的天气，它自己查不到，但你可以给它提供一个 `get_weather` 的函数说明书。

AI 看到你的需求和说明书后，不会直接回答天气，而是输出一段 JSON 代码：「我决定调用 `get_weather`，参数是 `深圳`」。 然后你的代码拿着这个参数去真正调用 API，查到结果后再喂给 AI，AI 最后总结成人类语言回复你。

划重点： Function Calling 是最基础的机制，它赋予了 AI 使用工具的「潜能」。

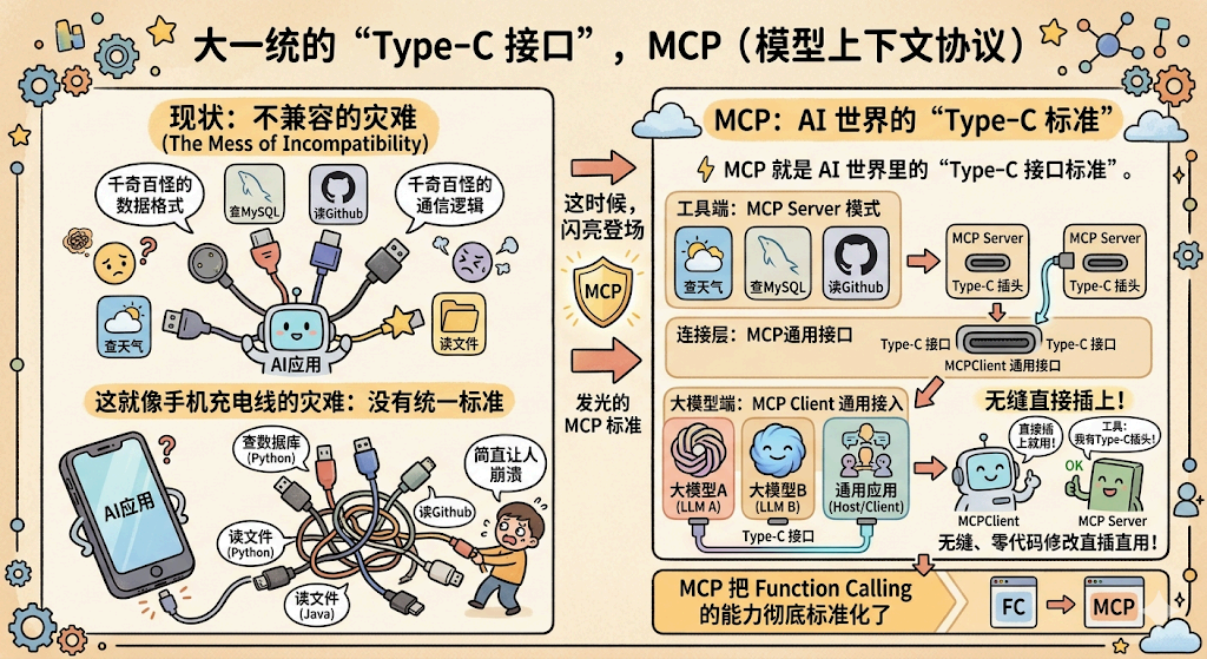
第二阶段：大一统的「Type-C 接口」 ， MCP (模型上下文协议)

有了 Function Calling，大家都很兴奋，开始给 AI 疯狂写各种工具（查天气、查 MySQL、读 Github 等等）。

但很快，新的灾难来了：**接口完全不兼容。**

张三用 Python 写了个查数据库的工具，李四用 Java 搞了个读本地文件的工具。由于没有统一的标准，你的 AI 应用想要接入这些现成的工具，就得挨个去适配它们千奇百怪的数据格式和通信逻辑。这就像你买了个新手机，却发现大家用的充电线有圆头、扁头、方头，简直让人崩溃。

这时候，**MCP（Model Context Protocol，模型上下文协议）** 闪亮登场。



用大白话说，MCP 就是 AI 世界里的 **「Type-C 接口标准」**。

它是一套通用的开源协议。只要外部工具（数据源、数据库、代码仓库）按照 MCP 的标准开发成 MCP Server，那么无论前端是哪个大模型（MCP Client），都可以**无缝、零代码修改**地直接插上去用。MCP 把 Function Calling 的能力彻底标准化了。

第三阶段：终极进化 ， Skills（技能）为什么会出现？

好，现在 AI 有了底层机制（Function Calling），也有了标准化的工具箱（MCP）。是不是就天下太平了？

并没有！随着咱们让 AI 干的活越来越复杂，一个极其痛点的问题暴露了出来：**每次让 AI 干活，沟通成本太高了！**

举个例子，假设你要让 AI 帮你写一份项目日报。你手里虽然已经有了 MCP 提供的「读取数据库」和「读取 Jira」的标准化工具，但你每次跟 AI 聊天，还是得像个唐僧一样反复叮嘱。

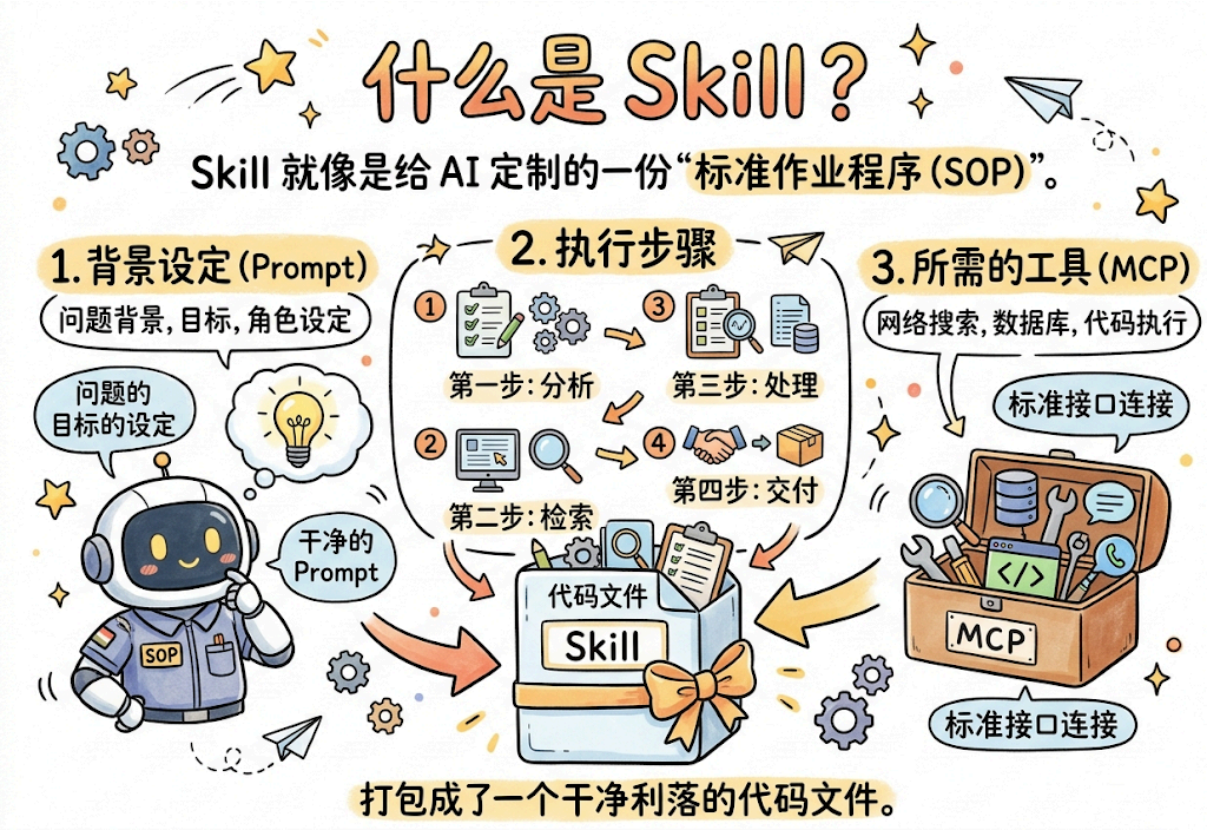
咱们来看看没有 Skills 之前的现状：

> **现状：每次对话都要重新描述相同的工作流程**
>
> 用户："帮我按XX格式生成报告"
> 用户："生成报告前，先去调用数据库工具获取今天的数据"
> 用户："记得要包含数据分析和总结部分"
> 用户："别忘了图表的排版细节..."
> （每次生成报告，都要重复这段痛苦的念经过程）

发现问题了吗？ 工具虽然标准化了，但「使用工具的流程和大脑的思考方式（Prompt）」并没有被沉淀下来。

为了解决这个痛点，**Skills（技能）** 应运而生！

什么是 Skill？ Skill 就像是给 AI 定制的一份 **「标准作业程序 (SOP)」**。它把解决特定问题所需的 **背景设定 (Prompt)**、**执行步骤** 和 以调用的资源（脚本、模板，或者通过 MCP 对接的外部工具），打包成了一个干净利落的代码文件。



咱们来看看用了 Skills 之后的方案：

> **Skills 方案：把经验沉淀为可复用的配置**
>
> ```
> ---
> name: report-generator
> description: 按照公司标准格式自动收集数据并生成报告
> tools:
>  - mcp-jira-reader      # 挂载所需的 MCP 工具
>  - mcp-database-query
> ---
>
> # 报告生成流程
>
> 1. 包含封面页（模板见 templates/cover.md）
> 2. 执行数据分析（自动调用 mcp-database-query 获取今日核心指标）
> 3. 提取任务进度（自动调用 mcp-jira-reader 获取任务状态）
> 4. 生成图表和摘要
> ...
> ```

只要有了这个 Skill 文件，你下次只需要对 AI 说一句：「**帮我生成今天的日报**」。 AI 就会自动加载 `report-generator` 这个技能，自动按顺序调用 MCP 工具，按规定格式输出。一步到位，神清气爽！

拆解：一个 Skill 的内部结构长什么样？

从上面的对比图咱们能看出来，一个优秀的 Skill，结构是非常清晰的，通常分为两部分：

1. **头部配置（Frontmatter）：** 通常用 YAML 语法写在最前面（被 `--` 包裹）。这里最关键的两个字段是 `name`（技能叫啥）和 `description`（技能干啥用的）。这俩字段后面你会看到，是 Agent 判断「要不要激活这个技能」的唯一依据，写得好不好，直接决定技能能不能被正确触发。如果技能执行过程中需要特定的外部工具（比如某个 MCP Server），也可以在这里额外声明。
2. **技能主体（Body）：** 这里写的是具体的指令、工作流（Workflow）、规则限制和模板。这里的大白话指令，就是在教大模型」拿到工具后，第一步干啥，第二步干啥」。

Skills 最大的亮点：渐进式披露

了解完 Skill 的内部结构，你可能会产生一个新疑问：**Agent 启动的时候，是不是要把所有 Skill 的内容一股脑全塞进去？**

咱们来算一笔账。假设你的 Agent 装了 50 个 Skill，每个 Skill 的完整指令大概 2000 个 token。如果全部塞进 System Prompt：

```
50 个 Skill × 2000 token = 10 万 token
```

这会带来三个灾难：

- **贵**：大模型按 token 收费，每次对话还没开始干活，光加载技能就烧掉 10 万 token 的钱
- **慢**：上下文越长，模型处理速度越慢，响应延迟直线上升
- **蠢**：这是最致命的，用户可能只是想让 AI 帮忙压缩一张图片，结果模型脑子里同时塞着「写日报」「查数据库」「发邮件」等 49 个完全无关的技能指令。无关信息太多，模型的注意力被严重稀释，反而干不好眼前这件事
- **结论：一股脑全塞进去，既浪费钱又影响质量。**

那怎么办？这就引出了 Skills 架构里最核心的设计思想，**渐进式披露（Progressive Disclosure）**。

什么是渐进式披露？

用一句话概括：**Agent 启动时只看「菜单」，需要时才「点菜」。**

这个概念其实借鉴自用户界面设计领域的经典原则：只展示当前任务需要的信息，其余全部隐藏。你打开手机设置，首页只显示「Wi-Fi」「蓝牙」「通知」几个大类，不会一上来就把所有子选项全摊开，那就是渐进式披露。

放到 Skills 里，意思就是：**不要把整本百科全书一次性翻开，而是像查字典一样，先看目录，再翻到需要的那一页。**

三层加载架构

Skills 的渐进式披露把信息分成了三个层级，每一层只在被需要时才加载：

第一层：元数据（Metadata），只看菜单

Agent 启动时，只把每个 Skill 的 **name**（名称）和 **description**（描述）加载到 System Prompt 里。

还记得咱们前面讲的 Skill 头部配置吗？就是那段 YAML：

```
---
name: report-generator
description: 按照公司标准格式自动收集数据并生成报告
---
```

每个 Skill 的元数据大约只占 **100 个 token**。50 个 Skill 全部加载元数据 = 5000 token。

打个比方，这就像去餐厅吃饭。菜单上只列了菜名和一句话简介：「宫保鸡丁，花生米配鸡丁，微辣」。你靠这些信息就能决定今天想吃什么，不需要把每道菜的详细菜谱全贴在墙上。

Agent 看到这份「菜单」之后，就知道自己有哪些技能可以用。当用户提出需求时，Agent 扫一眼菜单，判断哪个 Skill 跟当前任务相关。

第二层：完整指令（SKILL.md 正文），下单点菜

当 Agent 判断某个 Skill 和当前任务相关时，才去读取这个 Skill 的完整 SKILL.md 内容，把详细的工作流指令加载到上下文里。

继续餐厅的比方：你看完菜单决定点「宫保鸡丁」，服务员这才去后厨把这道菜的详细菜谱拿出来，放多少油、炒几分钟、什么时候放花生米，全部按步骤来。

这一层建议控制在 **5000 token 以内**，保证加载后不会让上下文太臃肿。

第三层：附属资源（脚本/模板/参考文档），按需取食材

SKILL.md 里可能会引用一些额外的文件：Python 脚本、参考文档、模板文件等。这些附属资源只在 Agent 执行任务的过程中，**真正需要用到的时候才去读取**。

还是餐厅的比方：厨师按菜谱做到第三步，发现需要松子，这才去仓库拿。不是一开始就把仓库里所有食材全搬到灶台上。

这一层的好处是：附属资源可以很大（几千行的脚本、整份参考手册），因为它们不会在 Agent 启动时占用上下文，只在真正执行时按需访问。

走一遍完整的加载过程

光说不练假把式，咱们用一个具体场景走一遍三层加载的完整流程。

假设你的 Agent 装了 30 个 Skill，其中有一个叫 `pdf-processor`，专门处理 PDF 文件。现在用户问了一句：**「帮我把这个 PDF 表单填了。」**

- **第一层启动：** Agent 启动时，30 个 Skill 的名称和描述已经在 System Prompt 里了（大约 3000 token）。Agent 扫一圈菜单，看到 `pdf-processor` 的描述是「处理 PDF 文件，支持解析、填表、格式转换」，跟用户的需求对上了。
- **第二层加载：** Agent 决定启用 `pdf-processor`，于是读取 `pdf-processor/SKILL.md` 的完整内容，获得详细的工作流指令。
- **第三层按需访问：** SKILL.md 里写着「填写表单时，请参考 `forms.md` 获取表单字段规则」。Agent 在执行到填表这一步时，才去读取 `forms.md`。

整个过程，Agent 只加载了 **1 个 Skill 的完整内容**，其余 29 个 Skill 自始至终只花了名称和描述的 token。

效果有多夸张？用数字说话

	全量加载	渐进式披露
启动时加载量	50 × 2000 = **10 万 token**	50 × 100 = **5000 token**
执行任务时追加	0（已经全部加载了）	1 × 2000 = **2000 token**
总消耗	**10 万 token**	**7000 token**
节省比例	—	**节省约 93%**

省了 93% 的 token，而且因为上下文里只有跟当前任务相关的信息，模型的注意力更集中，**响应质量反而更高**。

这就是为什么渐进式披露被认为是 Skills 架构最核心的创新，它让 Agent 可以拥有几十上百个技能，却不用为此付出巨额的上下文成本。技能装得越多，渐进式披露的优势就越明显。

灵魂拷问：Skills 不就是高级一点的 Prompt 吗？

可能看到这里，有同学心里犯嘀咕了：「我看你这 `SKILL.md` 里面，主体部分不还是写了一堆自然语言的步骤吗？这不就是个长一点的 Prompt（提示词）吗？跟我在对话框里敲字有什么区别？」

这个问题问得太好了！其实很多人刚接触时都会有这个错觉。咱们来掰扯掰扯它俩到底是什么关系、有什么区别：

- **从关系上看：包含与被包含。** Prompt 是 Skill 的「灵魂核心」，但不是全部。一个完整的 Skill = Prompt（指令与流程） + 挂载的工具列表（MCP） + 附属资源（脚本 / 模板 / 参考文档）。
- **从能力上看：动嘴与动手。** Prompt 只能控制大模型的「嘴」。你写一万字的 Prompt 教它怎么查数据库，它也只能给你输出一段怎么查的文本。而 Skill 给 AI 赋予了「手」。通过配置文件里绑定的 `tools`，AI 在阅读 Prompt 的同时，是真的能去后台调用代码、拉取数据的。
- **从工程上看：临时工与标准资产。** 你在对话框里敲的 Prompt 是「临时工」，上下文一长它就忘了，下次还得重敲。而 Skill 是一份写在项目目录里的配置文件（通常是 YAML 或 Markdown），它是可以提交到 Git 仓库里的**代码资产**。它把个人经验变成了整个团队都可以直接复用的 **标准作业程序（SOP）**。
- 打个粗俗点的比方：
- **Prompt** 是你站在厨房门口喊：「去给我炒个鱼香肉丝，先放葱姜蒜，再放肉丝！」
- **Skill** 是你把《鱼香肉丝菜谱》写好，连带厨房里自动炒菜机的【启动钥匙】，一起装进了一个信封里。以后谁饿了，拿着这个信封扫一下，菜就自动炒出来了。

实战加餐：以最近爆火的 Claude Code 为例，看看 Skills 到底有多强？

光说不练假把式。咱们就以它为例，看看真实项目里是怎么玩 Skills 的。

假设你在开发一个前端网站，经常需要把设计稿里的大图片压缩、转换成 WebP 格式，然后再挪到 `public` 文件夹里。如果每次都让大模型现写一段 Python 脚本来跑，既费时间又容易出错。

有了 Skills 之后，你可以直接在项目目录下建一个专门干这活的「图片优化技能」。

- **目录结构长这样：**

```
your-web-project/
├── src/
├── .claude/                    # Claude Code 的专属配置目录
```

```
│   └── skills/                  # 💡 这里就是技能大本营
│       └── image-optimizer/     # 我们自定义的「图片优化技能」
│           ├── SKILL.md         # 技能的说明书和工作流
│           └── optimize.py      # 真正干活的 Python 压缩脚本
```

- **SKILL.md 里面写了啥？** 咱们刚才说了，一个优秀的 Skill 分为「头部配置」和「技能主体」。在 Claude Code 里，它是这么写的：

```
---
name: image-optimizer
description: 专门用于将大尺寸图片压缩并转换为 webp 格式，输出到 public/assets 目录。
tools:
  - mcp-python-runner  # 声明需要用到运行 Python 代码的工具能力
---

# 任务流程
当你收到优化图片的指令时，请严格按照以下步骤执行：
1. 读取用户指定的原图片文件。
2. 调用同目录下的 `optimize.py` 脚本，将原图转换为高质量的 webp 格式。
3. 将转换后的图片移动到项目的 `public/assets/` 目录下。
4. 在终端打印出压缩前后的文件大小对比。
```

- **怎么使用？** 配置好之后，你在 Claude Code 的终端里，只需要像聊天一样发一句话（或者直接输入内置指令 `/image-optimizer`）：

> **用户：** "把这张刚生成的测试图，用你的 image-optimizer 技能处理一下。"

Claude 会自动扫描 `skills/` 目录，读取 `SKILL.md` 理解整个工作流，然后默默在后台调用脚本、压缩图片、移动文件，最后在终端里回复你：「搞定了！图片大小从 631KB 压缩到了 56KB，已经放在 public 目录啦！」

发现了吗？这种目录结构最大的好处是「热插拔」。**有了 Skill，你实际上是在给 AI「教流程」、「写 SOP」。**

以后再遇到类似的脏活累活，AI 就能像个熟练的老员工一样，直接照着这个目录里的 SOP 帮你把活干得漂漂亮亮！

总结：理清它们的三角关系

最后，我再用一句话帮大家总结一下今天聊的这三个核心概念：

- **Function Calling（函数调用）：** 是底层技术，让大模型拥有了「伸手」调取外部工具的**能力**。
- **MCP（模型上下文协议）：** 是接口标准，统一了全网工具的接入规格，让 AI 拥有了「Type-C」通用**插口**。
- **Skills（技能）：** 是上层的业务封装，它将 Prompt 指令和 MCP 工具组合在一起，按清晰的**目录结构**管理起来，变成了 AI 可以随拿随用、彻底告别重复劳动的「**标准工作流**」。
- 技术的发展永远是这样，从「能干活」（Function Calling），到「统一标准」（MCP），再到「工程化沉淀和管理」（Skills）。